1. ТЕОРЕТИЧНІ ОСНОВИ

Для опису так званого одновимірного масиву потрібно, крім його імені, вказати розмір та тип, що є спільним типом для всіх елементів масиву:

тип ім'я[розмір];

Тут тип – спільний тип елементів масиву;
ім'я – ім'я масиву;
розмір – кількість елементів масиву.

Звертаємо увагу на наступне:

- кількість елементів масиву задається тільки натуральною константою або додатним виразом над константами з натуральним значенням;
- типом параметру розмір може бути тільки цілочисловий тип;
- використання змінних для задавання кількості елементів заборонене;
- елементи масиву мають один і той самий тип (тип), причому цей тип може бути довільним.

Приклади опису масивів:

```
int a[20], b[100];
bool flags[10];
```

Іноді можливий опис масиву без вказівки кількості елементів:

```
extern unsigned long ULArray [ ];
```

Це означає, що даний масив зовнішній і йому відведена пам'ять в іншій частині програми.

Для звертання до елементу масиву вказують ім'я масиву й у квадратних дужках індекс, значення якого визначає розташування елементу всередині масиву. За значенням індексу визначається зсув елементу відносно адреси першого елементу масиву, який має нульовий зсув. Індексувати можна як константами і змінними, так і виразами, результат обчислення яких дає значення дискретного типу, яке автоматично перетворюється до цілочислового типу. Діапазон змінення індексу не може перевищувати 2 Гбайт (у випадку багатовимірних масивів це стосується кожного з вимірів). Таке ж саме обмеження має місце і для загального обсягу пам'яті, займаної масивом.

Елементи масиву завжди **нумеруються від нуля**. При цьому треба пам'ятати, що компілятор не перевіряє набуття індексом недопустимого значення (від'ємного або більшого номера останнього елементу). Це може призвести до помилок на етапі виконання програми. У багатьох випадках набуття індексом недопустимого значення може не призвести до аварійного завершення програми і, як слідство, програма просто дає невірний результат.

Масив може бути ініціалізований при визначенні, тобто одержати початкові значення елементів. Для цього після квадратних дужок в описі масиву перелічуються через кому у фігурних дужках значення, які присвоюються елементам масиву. При необхідності можна ініціалізувати тільки перші елементи. Якщо ініціалізуються всі елементи, то у визначенні масиву можна не задавати розмір. Наприклад:

```
float F[4];              // Визначення без ініціалізації
int I1[3] = {1, 2, 3 }, I2[] = {10, 20, 30};   // Ма-
                         // сиви із трьома елементами
long int LI[10] = {7, 2, 17};   // Значення отримують
                         // тільки перші три елементи
```

Розмір масиву може задаватися з використанням раніше оголошених констант. Можна також за допомогою специфікатора **typedef** попередньо описати масивний тип, а потім вживати його для опису масивів:

```
const int
  N = 100;
typedef
  double TDbArray[N];                    // Тип – масив
                  // Можливі такі методи опису масиву
  double A1[2 * N], A2[N];
  TDbArray A3;
```

Операція **sizeof**, застосована до імені масиву або імені масивного типу, повертає кількість байт, що відводиться під масив. Наприклад, для наведених вище описів, можливе виконання таких операторів:

```
int s1 = sizeof (A1);                    // s1 == 1600
int s2 = sizeof A2;                       // s2 == 800
int s3 = sizeof (TDbArray);               // s3 == 800
```

2. ПРИКЛАДИ ПРОГРАМ

Приклад 1. Дано натуральне число n і масив з n цілих чисел ($n \le 20$). Чи правда, що вміст масиву однаково читається в прямому і зворотному напрямках?

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
```

```
    int n;                          // Кількість елементів масиву
    int a[20];                                          // Масив
    bool flag = true;
    cout << "n = ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cout << "a[" << i << "] = ";
        cin >> a[i];
    }
    for (int i = 0, j = n - 1; i < j; i++, j--)
        if (a[i] != a[j])
        {
            flag = false;
            break;
        }
    if (flag)
        cout << "Yes";
    else
        cout << "No";
    cout << "Press any key";
    _getch();
    return 0;
}
```

Можна запропонувати декілька варіантів програм для розв'язання сформульованої задачі, які дещо відрізняються від наведеної вище. Так, у наступній програмі використовується тільки одна змінна для індексування:

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    int n;                          // Кількість елементів масиву
    int a[20];                                          // Масив
    bool flag;
    cout << "n = ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cout << "a[" << i << "] = ";
        cin >> a[i];
    }
    for (int i = 0, flag = true; (i < n / 2) && flag; i++)
```

```
        if (a[i] != a[n - 1 - i])
            flag = false;
    if (flag)
        cout << "Yes";
    else
        cout << "No";
    cout << "Press any key";
    _getch();
    return 0;
}
```

Приклад 2. Які різні цифри входять у ціле число *N*?

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    long int N;                              // Не більше 10 цифр!
    int Numerals[10],                                 // Різні цифри
    QuantityOfNumerals,                  // Кількість різних цифр
    Numeral;                                  // Виділювана цифра
    bool Flag;
    cout << "N = ";
    cin >> N;
    cout << "Number " << N
         << " contains such various numerals: \n";
    QuantityOfNumerals = 0;       // Поки що немає жодної цифри
    do
    {
        Numeral = abs(N % 10);          // Одержуємо нову цифру
        Flag = false;//Вважаємо, що цифра раніше не виділялася
     // і в циклі перевіряємо, чи виділялася раніше така цифра
        for (int i = 0; i < QuantityOfNumerals; i++)
            if (Numeral == Numerals[i])
            {
                Flag = true;     // Раніше така цифра вже була!
                break;                          // Вихід з циклу
            }
        if (Flag == false)             // Краще так: if (!Flag)
        {
          Numerals[QuantityOfNumerals] = Numeral;
            QuantityOfNumerals++;
        }
```

```
        N = N / 10; // Відтинаємо останню цифру в запису числа
    } while (N != 0);
    for (int i = 0; i < QuantityOfNumerals; i++)
        cout << Numerals[i] << '\n';
    cout << "Press any key";
    _getch();
    return 0;
}
```

Зауважимо, що в програмі цифри виділяються, починаючи від самої правої з них у запису числа. Аналогічно вони і виводяться. Щоб змінити порядок виведення на протилежний, останній цикл **for** потрібно записати так:

```
    for (int i = QuantityOfNumerals -1; i >= 0; i--)
        cout << Numerals[i] << '\n';
```

Значно простішою є така програма:

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    long int N;                              // Не більше 10 цифр!
    int Amount[10];                    // Кількість входжень цифр
    cout << "N = ";
    cin >> N;
    cout << "Number " << N
         << " contains such various numerals: \n";
    for (int i = 0; i < 10; i++) // Поки що вважаємо, що цифри
        Amount[i] = 0;                       // не входять у число
    do
    {
        Amount[abs(N % 10)]++;
        N = N / 10; // Відтинаємо останню цифру в запису числа
    } while (N != 0);
    for (int i = 0; i < 10; i++)
        if (Amount[i] > 0)
            cout << i << '\n';
    cout << "Press any key";
    _getch();
    return 0;
}
```

Приклад 3. Упорядкувати числовий масив, що вміщує не більш 20 дійсних чисел, за неспаданням методом «бульбашки». Дано натуральне число *n*

($n \le 20$) і послідовність дійсних чисел a_1, a_2, …, a_n. Упорядкувати члени послідовності за неспаданням методом «спливання» («бульбашки»). Сутність методу полягає в наступному: для будь-якого значення i з ряду 1, 2, ..., n – 1 послідовно розглядаються всі члени a_j, $j = n – 1, n – 2, ..., i – 1$, і при виконанні умови $a_j < a_{j+1}$ члени a_j і a_{j-1} міняються місцями з переходом до нового значення j, незалежно від того, переставлялися числа чи ні.

Розв'язок.

```
#include <iostream>
#include <conio.h>

using namespace std;
int main()
{
    int N;                          // Кількість елементів масиву
    double a[20];                                       // Масив
    double b;
    cout << "N = ";
    cin >> N;
    for (int i = 0; i < N; i++) {
        cout << "a[" << i << "] = ";
        cin >> a[i];
    }
    cout << "\nInitial array\n";    // Виводимо початкові дані
    for (int i = 0; i < N; i++)
        cout << a[i] << " ";
    for (int i = 0; i < N - 1; i++)       // Метод "бульбашки"
        for (int j = N - 1; j > i; j--)
            if (a[j] < a[j - 1])
            {
                b = a[j];
                a[j] = a[j - 1];
                a[j - 1] = b;
            }
    cout << "\nResultant array\n";
    for (int i = 0; i < N; i++) // Виводимо результуючий масив
        cout << a[i] << " ";
    cout << "\nPress any key";
    _getch();
    return 0;
}
```

Приклад 4. Дано натуральне число n ($n \le 20$), дійсне число w і масив з n дійсних чисел. Видалити з масиву елемент, що є найближчим до числа w.

Розв'язок.

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{
    double a[20], w;
    int N;
    int k;                  // Номер елементу, що видалятиметься
    double Min;                             // Мінімальна відстань
    cout << "w = "; cin >> w;
    cout << "N = ";
    cin >> N;
    for (int i = 0; i < N; i++)
    {
        cout << "a[" << i << "] = ";
        cin >> a[i];
    }
    cout << "\nInitial array\n";     // Виводимо початкові дані
    for (int i = 0; i < N; i++)
        cout << a[i] << " ";
    k = 0;
    Min = abs(w - a[0]);
    for (int i = 1; i < N; i++)          // Перебираємо елементи
        if (fabs(w - a[i]) < Min)
        {
            k = i;              // Запам'ятовуємо номер елементу
            Min = fabs(w - a[i]);       // та відстань до нього
        }
    for (int i = k + 1; i < N; i++)
        a[i - 1] = a[i];                      // Зсув елементів
    N--;                                // Розмір масиву зменшився
    cout << "\nResultant array";
    if (N == 0)
        cout << " is empty";
    else
    {
        cout << ":\n";
        for (int i = 0; i < N; i++)    // Виводимо результуючий
            cout << a[i] << " ";                          // масив
    }
    cout << "\nPress any key";
    _getch();
```

```
    return 0;
}
```

Приклад 5. Дано натуральне число n $(n \le 100)$ і масив з n дійсних чисел. Визначити в цьому масиві кількість ділянок, на яких його елементи убувають.

Розв'язок.

```
#include <iostream>
#include <conio.h>

using namespace std;
int main()
{
    int n, i;                       // Кількість елементів масиву
    double a[100];                                      // Масив
    int c;                                 // Кількість ділянок
    bool flag;
    cout << "n = ";
    cin >> n;
    for (i = 0; i < n; i++)
    {
        cout << "a[" << i << "] = ";
        cin >> a[i];
    }
    c = 0;
    flag = true;
    for (i = 1; i < n; i++)     // Починаємо з другого елементу
        if (a[i] > a[i - 1])                          // Є зростання
        {
            if (flag)                               // Нова ділянка
            {
                c++;
                flag = false;
            }
        }
        else                                          // Незростання
            flag = true;
    cout << "Count = " << c << '\n';
    cout << "Press any key";
    _getch();
    return 0;
}
```

3. ВАРІАНТИ ЗАДАЧ

1. Сформувати послідовність цілих чисел a_1, a_2, …, a_{50}, де $a_1 = 1$; $a_2 = 1$; $a_i = a_{[i/2]} + a_{i-2}$ $(i = 3, 4, \ldots, 50)$.
2. Дано натуральне число n $(n \le 15)$ і дійсні числа a_1, a_2, …, a_n, b_1, b_2, …, b_n. Обчислити
$$(a_1 + b_n) \cdot (a_2 + b_{n-1}) \cdot \ldots \cdot (a_n + b_1).$$
3. Дано натуральне число n $(n \le 100)$ і дійсні числа a_1, a_2, …, a_n. Знайти номер найменшого додатного числа в заданій послідовності. Якщо в ній відсутні додатні числа, відповіддю повинне бути значення –1.
4. Дано натуральне число n $(n \le 100)$. Одержати всі прості числа, що не перевищують це значення. Для прискорення обчислень при перевірці поточного числа на його простоту можна використати раніше знайдені прості числа.
5. Дано натуральне число n $(n \le 100)$ і цілі числа a_1, a_2, …, a_n. З'ясувати, чи є серед чисел a_1, a_2, …, a_n співпадаючі.
6. Дано натуральне числа n $(n \le 100)$ і послідовність цілих чисел a_1, a_2, …, a_n. Поміняти місцями в цій послідовності найбільший і найменший члени. Якщо в послідовності кілька найбільших або найменших елементів, то розглядати останні з таких.
7. Дано натуральне число n $(n \le 100)$ і послідовність дійсних чисел x_1, x_2, …, x_n. Якщо в результаті заміни від'ємних членів послідовності x_1, x_2, …, x_n їхніми квадратами члени нової послідовності будуть утворювати незростальну послідовність, то одержати добуток членів первинної послідовності; у противному випадку одержати їхню суму.
8. Дано натуральне число n $(n \le 100)$ і масив з n дійсних чисел. Кожен елемент цього масиву замінити середнім арифметичним усіх передуючих йому елементів, включаючи його самого.
9. Дано натуральне число n $(n \le 100)$ і послідовність з n дійсних чисел. Вивести ділянку послідовності, починаючи з максимального члена і закінчуючи мінімальним, поза залежністю від їх взаємного розташування (завжди першим повинен виводитись максимальний член, а останнім – мінімальний).
10. Дано масив з n цілих чисел, де n – задане натуральне число $(n \le 100)$. Сформувати новий масив, що містить тільки ті елементи первинного масиву, які є простими числами.

11. Дано натуральне число n ($n \le 100$), ціле число m і масив з n цілих чисел. Чи міститься число m у масиві? Відповіддю повинне бути число 1 при позитивній відповіді і число 0 – при негативній.
12. Дано натуральне число n ($n \le 10$) і цілі числа a_1, a_2, …, a_{3n}. З'ясувати, чи правда, що для всіх a_{n+1}, a_{n+2}, …, a_{3n} є рівні серед a_1, a_2, …, a_n.
13. Дано натуральне число n ($n \le 100$) і послідовність цілих чисел a_1, a_2, …, a_n. Одержати нову послідовність, вилучивши з первинної послідовності всі члени зі значенням $\min(a_1, a_2, \ldots, a_n)$.
14. Дано натуральне число n ($n \le 100$) і масив з n цілих чисел. Знайти найменше натуральне число, відсутнє в масиві.
15. Дано натуральне число n ($n \le 100$) і послідовність відмінних від нуля цілих чисел a_1, a_2, …, a_n. Якщо в послідовності числа з різними знаками чергуються, вивести початкову послідовність. У противному випадку виключити з послідовності всі додатні члени.
16. Дано натуральне число n ($n \le 100$) і послідовність дійсних чисел a_1, a_2, …, a_n. Залишити без зміни послідовність a_1, a_2, …, a_n, якщо вона впорядкована за неубуванням або незростанням; у противному випадку видалити з неї ті члени, порядкові номери яких кратні п'яти, зберігши порядок членів, що залишилися.
17. Дано натуральне число n ($n \le 100$) і масив дійсних чисел a_1, a_2, …, a_n. Упорядкувати цей масив за неубуванням. Скористатися таким методом. Знайти найменший елемент масиву і переставити його з першим елементом, потім серед елементів масиву, починаючи з другого, знайти найменший і переставити його з другим і т. д.
18. Дано натуральне число n ($n \le 100$) і цілі числа a_1, a_2, …, a_n. Якщо в послідовності a_1, a_2, …, a_n жодне парне число не розташоване після непарного, то одержати нову послідовність з усіх від'ємних членів вихідної послідовності. У противному випадку в нову послідовність умістити всі додатні числа з послідовності a_1, a_2, …, a_n. У обох випадках порядок проходження чисел замінити на зворотний.
19. Дано натуральне число n ($n \le 100$) і дійсні числа a_1, a_2, …, a_n. Без використання додаткового масиву переставити члени послідовності a_1, a_2, …, a_n так, щоб спочатку розташувалися всі її невід'ємні члени, а потім всі від'ємні. Зберегти початковий порядок, як серед всіх невід'ємних членів, так і серед від'ємних.

    Дано натуральне число n ($n \le 100$), ціле число k ($k = \pm 1$) і послідовність цілих чисел a_1, a_2, …, a_n. Здійснити циклічний зсув елементів послідовності на 1 позицію вправо (при $k = 1$) або вліво (при $k = -1$).